

– הרצאה 10 Express

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

1. מבוא ל-Express
2. Express routing
3. Express - Request, Response



1. מבוא ל-Express

מה זה Express?

- Express is a minimal and flexible Node.js web application framework that provides a robust set of features to develop web and mobile applications

● אקספרס מספקת מודול קליל שעוטף את הפונקציונליות של מודול http של Node

● מאפשרת לתפעל בקלות:

- נתיבים Routes
- HTTP Responses
- HTTP Requests
- Cookies
- ועוד

התקנת Express – דוקומנטציה

דוקומנטציה: <https://expressjs.com/en/5x/api.html>



Express

search

HomeGetting startedGuideAPI referenceAdvanced topicsResources

express()

Application

Properties

app.locals

app.mountpath

app.router

Events

mount

Methods

app.all()

app.delete()

app.disable()

app.disabled()

app.enable()

app.enabled()

app.engine()

app.get()

app.get()

app.listen()

app.METHOD()

app.param()

app.path()

app.post()

app.put()

app.render()

app.route()

app.set()

app.use()

Request

Response

Router

Methods

app.all(path, callback [, callback ...])

This method is like the standard `app.METHOD()` methods, except it matches all HTTP verbs.

Arguments

Argument	Description	Default
path	The path for which the middleware function is invoked; can be any of: - A string representing a path. - A path pattern. - A regular expression pattern to match paths. - An array of combinations of any of the above. For examples, see Path examples.	'/' (root path)
callback	Callback functions; can be: - A middleware function. - A series of middleware functions (separated by commas). - An array of middleware functions. - A combination of all of the above. You can provide multiple callback functions that behave just like middleware, except that these callbacks can invoke <code>next('route')</code> to bypass the remaining route callback(s). You can use this mechanism to impose pre-conditions on a route, then pass control to subsequent routes if there is no reason to proceed with the current route. When a callback function throws an error or returns a rejected promise, <code>next(err)</code> will be invoked automatically. Since router and app implement the middleware interface, you can use them as you would any other middleware function. For examples, see Middleware callback function examples.	None

הרמת שרת Express



- נשתמש בפונקציה `app.listen`
- דוקומנטציה: <http://expressjs.com/en/5x/api.html#app.listen>
- כדי ליישם את אקספרס בתור שרת ה-HTTP של אפליקציית Node, צריך ליצור מופע של אקספרס ולהאזין ל-`port`:

```
const express = require('express');  
const app = express();  
app.listen(8080);
```
- שורה 3 מקשרת (`binds`) בין חיבור HTTP לבין `port`.
- חיבור ה-HTTP זהה לחיבור שנוצר על ידי `server.listen()` בשימוש במודול `http`
- לכן, הקוד הבא מחליף את הקודם: `http.createServer(app).listen(8080)` (ניתן לראות בעמוד הבא השוואה)

הרמת שרת Express - המשך

● השוואה בין הרמת שרת ע"י המודול http (שלא נלמד ונשתמש בקורס זה) לבין שימוש ב-Express

JS index.js x

JS index.js ▸ ...

```
1 const express = require('express');
2 const app = express();
3
4 app.listen(8080);
```

JS index.js x

JS index.js ▸ ...

```
1 const http = require("http");
2
3 http.createServer((req, res) => {}).listen(8080);
```



Express routing .2

Routes – overview

ב-Express ישנן פונקציות מוכנות שעוזרות לנו לטפל בראוטים, כך שנוכל לממש CRUD מלא:

- app.get
- app.post
- app.put
- app.delete
- app.all

JS index.js x

JS index.js ▸ ...

```
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get('/', (req, res) => res.send('Server root'));
6  app.get('/login', (req, res) => res.send('Login page'));
7  app.post('/contact', (req, res) => res.json({firstname: "Alf"}));
8  app.put('/contact', (req, res) => res.send('Update contact'));
9  app.delete('/contact', (req, res) => res.send('Delete contact'));
10
11 app.all('*', (req, res) => res.send('Global handler for all routes'));
12
13 app.listen(port);
14 console.log(`listening on port ${port}`);
```

app.get

get app תופס איורע בקשה שמגיעה בשיטת

```
JS index.js x
JS index.js ▸ ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get('/', (req, res) => {
6    res.send(`<!doctype html>
7      <html>
8        <head></head>
9        <body>
10         <h1>Wow</h1>
11        </body>
12      </html>`);
13  });
14
15  app.listen(port);
16  console.log(`listening on port ${port}`);
```

app.get(path, callback [, callback ...])

Routes HTTP GET requests to the specified path with the specified callback functions.

Example

```
app.get('/', function (req, res) {
  res.send('GET request to homepage')
})
```

← → ↻ ⓘ localhost:8080

Wow

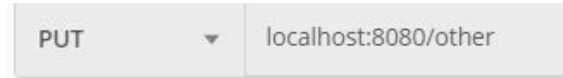
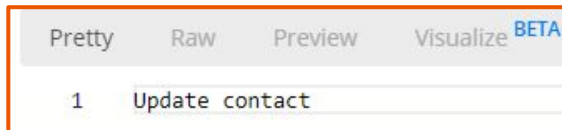
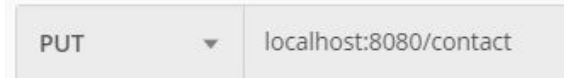
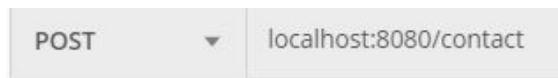
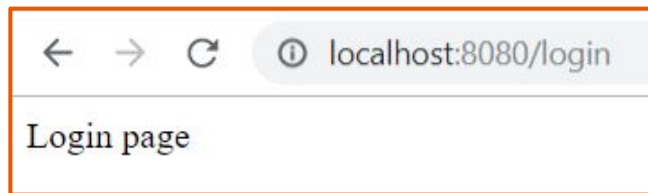
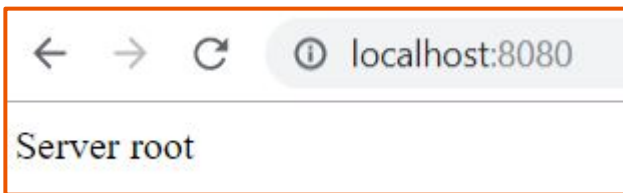
תרגיל - Routes

```
5 app.get('/', (req, res) => res.send('Server root'));
6 app.get('/login', (req, res) => res.send('Login page'));
7 app.post('/contact', (req, res) => res.json({firstname: "Alf"}));
8 app.put('/contact', (req, res) => res.send('Update contact'));
9 app.delete('/contact', (req, res) => res.send('Delete contact'));
10
11 app.all('*', (req, res) => res.send('Global handler for all routes'));
```

מה יודפס בכל אחד מהמצבים הבאים ? ●

- GET localhost:8080
- GET localhost:8080/login
- POST localhost:8080/contact
- PUT localhost:8080/contact
- PUT localhost:8080/other
- PATCH localhost:8080/contact

תרגיל - תשובות - Routes



למה?

חוקיות סדר קריאת ה-routes

```
app.all('*', (req, res) => {  
  res.send("Global handler");  
});  
app.get('/', function(req, res) {  
  res.send("Server root");  
});
```

כל טיפול ב- route הוא כמו if, else

1. לאן נכנס הקוד? ל- all או ל- get? ומה יקרה אם נחליף את הסדר?
2. כיצד נעצרת שרשרת של callbacks?
3. כיצד נמנע מהשרשרת להיעצר?
4. (מה ההבדל בין צורת הכתיבה של הפונקציה האנונימית בחלק הראשון לעומת החלק השני?)

חוקיות סדר קריאת ה-routes - המשך

- הקוד יכנס רק ל- route הראשון שהוא קורא ומתאים. במקרה זה, ייכנס ל- all. אם נחליף, הוא יכנס רק ל- ./.
 - אם שני routes מתאימים - צריך לנהל בעזרת middleware.
 - Middlewares הם callbacks "ביניים" בין ה- request לבין ה- response, הם יוצרים שרשרת callbacks.
 - את הסדר מגדירים על ידי הוספת פרמטר next וקריאה לפונקציה next()
- אחרי הרצת ה- callback הראשון all, הקוד ייעצר.
אלא אם יש התאמה נוספת של routes ומוגדרת שרשרת callbacks

Middleware

- Middleware = Code that sits between two layers of software, or between OS and software

● ב-Express נעשה שימוש ב-middleware בין ה- request לבין ה- response
● דוקומנטציה: <https://expressjs.com/en/guide/using-middleware.html>

This example shows a middleware function mounted on the `/user/:id` path. The function is executed for any type of HTTP request on the `/user/:id` path.

```
app.use('/user/:id', function (req, res, next) {  
  console.log('Request Type:', req.method)  
  next()  
})
```



Middleware

- כדי לנהל שרשראות routes, צריך middleware

- נוסף פרמטר next וקריאה לפונקציה next()

JS index.js x

JS index.js ▶ ...

```
1 const express = require('express');
2 const app = express();
3 const port = process.env.PORT || 8080;
4
5 app.all("/*", (req, res, next) => {
6   console.log("1- All route");
7   next();
8 });
9 app.get("/", (req, res) => {
10   console.log("2- Get route");
11   res.send({result: "WOW!"});
12 });
13
14 app.listen(port);
15 console.log(`listening on port ${port}`);
```

Administrator: Windows PowerShell

```
PS C:\Users\OWNER\Desktop\Node> node index.js
listening on port 8080
1- All route
2- Get route
```

← → ↻ ⓘ localhost:8080

```
{"result": "WOW!"}
```

Routes parameters



- כדי למנוע יותר מדי הגדרות נתיבים, שולחים פרמטרים
- קישור: <http://expressjs.com/en/guide/routing.html>
- אופציות לשליחת פרמטרים:
 - Querystring
 - Path params
 - Body params

Routes querystring params

● אילו פרמטרים השרת מצפה לקבל ?

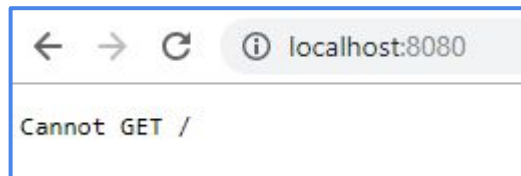
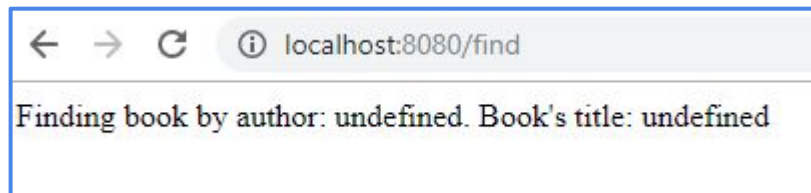
```
JS index.js x
JS index.js ▶ app.get("/find") callback
1  const express = require('express');
2  const url = require('url');
3  const app = express();
4  const port = process.env.PORT || 8080;
5
6  app.get("/find", (req, res) => {
7    const urlPart = url.parse(req.url, true);
8    const query = urlPart.query;
9
10   res.send(`Finding book by author: ${query.author}. Book's title: ${query.title}`);
11 });
12
13 app.listen(port);
14 console.log(`listening on port ${port}`);
```

← → ↻ ⓘ localhost:8080/find?author=Stephen%20King&title=it

Finding book by author: Stephen King. Book's title: it

● שימו לב - אפשר להשתמש גם ב-req.query בשביל לקבל את ה-querystring.

Routes querystring params



הבנת שגיאות:



Route url params

- איך נשלח פרמטרים דינמיים כחלק מה- ?route
- בדוגמה הבאה התבנית ב-url היא /user/:userId כאשר userId הוא פרמטר
- הפרמטר userId הוא פרמטר של האובייקט req.params

JS index.js x

JS index.js ▶ ...

```
1 const express = require('express');
2 const app = express();
3 const port = process.env.PORT || 8080;
4
5 app.get("/user/:userId", (req, res) => {
6   res.send(`Showing results for user: ${req.params.userId}`);
7 });
8
9 app.listen(port);
10 console.log(`listening on port ${port}`);
```

← → ↻ ⓘ localhost:8080/user/5

Showing results for user: 5

שליחת פרמטרים ב-POST

- 
- כששולחים פרמטרים ב-POST, הם נשלחים בגוף ההודעה, ב-body.
 - מגרסה 4 המודול body-parser מובנה באקספרס. החיבור מתבצע על ידי:
 - `app.use(express.json());`
 - הוא קורא את ה-body של ההודעה, שם נמצאים הפרמטרים שנשלחו מהקליינט
 - בדיפולט req.body הוא undefined אלא אם משתמשים ב-middleware כמו שאנו עושים כאן
 - **קוד ישן משתמש במודול body-parser, אם תראו דוגמאות כאלו – לא להשתמש!**

שליחת פרמטרים ב-POST

פרמטרים נשלחים בגוף ההודעה

```
JS index.js x
JS index.js ▸ app.post("/users") callback
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.use(express.urlencoded({extended: true}));
6  app.use(express.json());
7
8  app.post("/users", (req, res) => {
9    console.log(`Post request: username is ${req.body.username}`);
10
11    res.status(200).json({username: req.body.username});
12  });
13
14  app.listen(port);
15  console.log(`listening on port ${port}`);
```

שליחת פרמטרים ב-POST

express.urlencoded()

The screenshot shows the Postman interface for a POST request to `localhost:8080/users`. The **Body** tab is selected, and the `x-www-form-urlencoded` type is chosen. A table with two columns, **KEY** and **VALUE**, is visible. The first row has `username` as the key and `Shlomo Artzi` as the value. Below the table, the **Body** tab is selected, and the `JSON` format is chosen. The raw body content is shown as:

```
1 {  
2   "username": "Shlomo Artzi"  
3 }
```

express.json()

The screenshot shows the Postman interface for a POST request to `localhost:8080/users`. The **Body** tab is selected, and the `raw` type is chosen. The `JSON` format is selected. The raw body content is shown as:

```
1 {  
2   "username": "Shlomo Artzi"  
3 }
```



Express - .3 request, response

The Request object

Request הוא הפרמטר הראשון שעובר ל- callback של פונקציית ה- route



```
app.get("/user/:userId", (req, res) => {  
  console.log(`Param url: ${req.originalUrl}`);  
  res.send(`Showing results for user: ${req.params.userId}`);  
});
```

מאפיינים ומתודות של ה- http request object (לא בולם)

נתייחס לכתובת: <http://www.somedomain.co.il/resources/reg?ddd>

- originalUrl: [resources/reg?ddd](#)
- path: [reg](#)
- baseUrl: [resources](#)
- host
- ip: של הבקשה
- protocol
- method
- query
- get (header): מתודה שמחזירה את ההאדר של הבקשה
- headers: אובייקט

The Request object – parameters

```
JS index.js x
JS index.js ▸ ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/find", (req, res) => {
6      console.log(`Protocol:\t: ${req.protocol}`);
7      console.log(`Original url:\t: ${req.originalUrl}`);
8      console.log(`IP:\t: ${req.ip}`);
9      console.log(`Path:\t: ${req.path}`);
10     console.log(`Method:\t: ${req.method}`);
11     console.log(`Query:\t: ${JSON.stringify(req.query)}`);
12     console.log(`Connection:\t: ${req.get("connection")}`);
13     console.log(`Headers:\t: ${JSON.stringify(req.headers, null, 2)}`);
14
15     res.send("User request");
16 });
17
18 app.listen(port);
19 console.log(`listening on port ${port}`);
```

Administrator: Windows PowerShell

```
PS C:\Users\OWNER\Desktop\Node> node index.js
listening on port 8080
Protocol:      : http
Original url:   : /find?name=hello&age=18
IP:            : ::1
Path:          : /find
Method:        : GET
Query:         : {name:"hello","age":"18"}
Connection:    : keep-alive
Headers:       : {
  "host": "localhost:8080",
  "connection": "keep-alive",
  "pragma": "no-cache",
  "cache-control": "no-cache",
  "upgrade-insecure-requests": "1",
  "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/77.0.3865.90 Safari/537.36",
  "sec-fetch-mode": "navigate",
  "sec-fetch-user": "?1",
  "accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3",
  "sec-fetch-site": "none",
  "accept-encoding": "gzip, deflate, br",
  "accept-language": "en-US,en;q=0.9,he;q=0.8",
  "cookie": "ai_user=tYQPEi2019-09-02T11:40:25.940Z; sLastEmail=admin; sLastLang=en-gb; sLastLocale=en-gb; KeepLoggedIn=true; __LOTKEY__=00ae66da-eab2-4d52-9275-9b4ad3fa442e; __impid__=9Q5+HS1aKb92UhrYE90Rdw="
}
```

The Response object

Response הוא הפרמטר השני שעובר ל- callback של פונקציית ה- route
מאפשר לשלוח תגובה ללקוח



```
app.get("/user/:userId", (req, res) => {  
  console.log(`Param url: ${req.originalUrl}`);  
  res.send(`Showing results for user: ${req.params.userId}`);  
});
```

res.status

חשוב לשלוח את הסטטוס של התגובה, לא רק כשהסטטוס שונה מ- 200

- `res.status(200) //ok`
- `res.status(301) //permanent redirect`
- Http status codes – <https://www.restapitutorial.com/httpstatuscodes.html>



res.send



```
res.send('<html><body>hi</body></html>');
```

שליחת תגובה - מסוג buffer, string, object or array

דוגמה:

בשהמתודה send() מסתיימת, מוגדרים:

res.finished - משתנה בוליאני, סטטוס התגובה

res.header - אובייקט ההאדר

חשוב! אחרי פונקציה זו תוחזר תשובה והקוד לא ימשיך בצד השרת

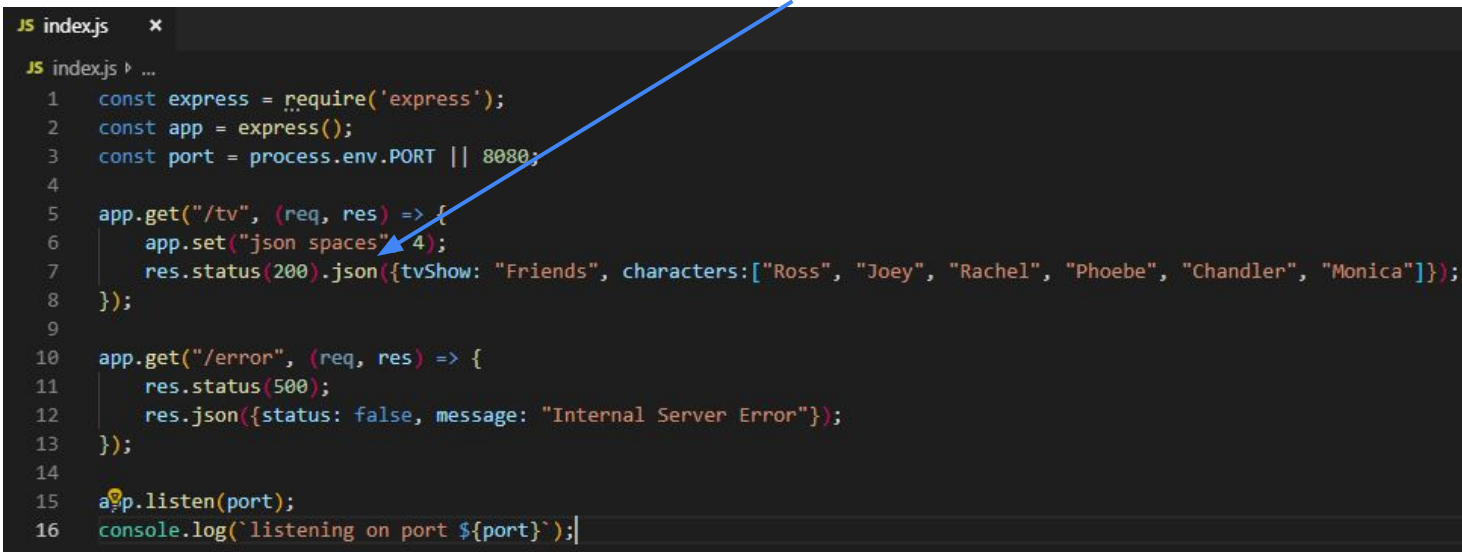
המתודה בצורה אוטומטית קובעת את סוג ה-header, למשל אובייקט json יחזיר האדר מסוג application/json

res.json

● אובייקט JS מומר למחרוזת JSON שנשלח בתגובה ללקוח

- `res.json(status, [object])`
- `res.json([body])`

● אפשר לשרשר פונקציות של `res`, כמו בראוט הראשון



```
JS index.js x
JS index.js ▸ ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/tv", (req, res) => {
6    app.set("json spaces", 4);
7    res.status(200).json({tvShow: "Friends", characters:["Ross", "Joey", "Rachel", "Phoebe", "Chandler", "Monica"]});
8  });
9
10 app.get("/error", (req, res) => {
11   res.status(500);
12   res.json({status: false, message: "Internal Server Error"});
13 });
14
15 app.listen(port);
16 console.log(`listening on port ${port}`);
```

המשך - res.json

```
localhost:8080/tv
{
  "tvShow": "Friends",
  "characters": [
    "Ross",
    "Joey",
    "Rachel",
    "Phoebe",
    "Chandler",
    "Monica"
  ]
}
```

● טיפול בשגיאות בצד השרת:

```
localhost:8080/error
{"status":false,"message":"Internal Server Error"}
```

× Headers Preview Response Cookies Timing

▼ General

Request URL: http://localhost:8080/tv
Request Method: GET
Status Code: 200 OK
Remote Address: [::1]:8080
Referrer Policy: no-referrer-when-downgrade

▼ Response Headers view source

Connection: keep-alive
Content-Length: 159
Content-Type: application/json; charset=utf-8
Date: Sat, 28 Sep 2019 16:23:00 GMT
ETag: W/"9f-Dn7yLYrCL06mcMFYdPJmg8vQ89s"
X-Powered-By: Express

► Request Headers (13)

Name	× Headers Preview Response Cookies Timing
error	▼ General Request URL: http://localhost:8080/error Request Method: GET Status Code: 500 Internal Server Error Remote Address: [::1]:8080 Referrer Policy: no-referrer-when-downgrade
in-page-script.js	▼ Response Headers view source Connection: keep-alive Content-Length: 50 Content-Type: application/json; charset=utf-8 Date: Sat, 28 Sep 2019 16:21:59 GMT ETag: W/"32-pqQZFhJrOGKR1LE57C0Xbr5m1Ic" X-Powered-By: Express
bmdblncegkenkacieihfhpjppoconhi/scripts	► Request Headers (13)
engines.json	
fheoggkfdfchfphceiefdbepaoioicaho/json	
content.css	
liecbbdmkiihnedobmlmillhodjkdmdb/css	